

← Main menu

Empower Gemini to take action with function calling

Course · 3 hours 30 minutes

Complete

Course overview

Empower Gemini to take action with function calling

Introduction to Function Calling with Gemini

Use function calling and grounding with Google Search to create a research tool

Enhance Gemini with access to external services with function calling: Challenge Lab

Your Next Steps

Course Badge

Course > Empower Gemini to take action with function calling >

Lab setup instructions and requirements

End Lab00:39:30

Caution: When you are in the console, do not deviate from the lab instructions. Doing so may cause your account to be blocked. [Learn more.](#)

Open Google Cloud Console

Username

student-02-c4ee8db87cb2l

Password

psZFmUD107pN

GCP Project ID

qw1k1abs-gcp-08-c85b3fb

Enhance Gemini with access to external services with function calling: Challenge Lab

Lab1 hourNo costIntermediate

★★★★☆Rate Lab

This lab may incorporate AI tools to support your learning.

Lab instructions and tasks25/100

GENAI067

Challenge Lab Overview

Objective

Related learning materials

Setup

Challenge Scenario

Your Challenge

Task 1. Open a Jupyter notebook in Vertex AI Workbench

Task 2. Define functions in Python

Task 3. Prompt the model to call your functions

Congratulations!

Enhance Gemini with access to external services with function calling: Challenge Lab

GENAI067

Challenge Lab Overview

This lab will challenge you to perform actions and automation across products. Instead of following step-by-step instructions, you are given a common business scenario and a set of tasks - you figure out how to complete them on your own! An automated scoring system (shown on this page) provides feedback on whether you have completed your tasks correctly.

When you take a Challenge Lab, you will not be taught Google Cloud concepts. You will need to use your skills to assess how to build the solution to the challenge presented. This lab is only recommended for students who have those skills. Are you up for the challenge?

Objective

This lab tests your ability to implement the **ReAct** pattern by extending LLMs to access external information, such as databases, third party services and a coding environment. This lab uses the following technologies and Google Cloud services:

- Vertex AI
- Vertex AI Workbench
- Gemini
- Function Calling

Related learning materials

If you are looking for resources that will help you develop the skills required to complete this lab successfully, consider reviewing relevant sections from the following:

Category	Resources
Learning Labs	- Introduction to Function Calling with Gemini - Use function calling and grounding with Google Search to create a research tool
Documentation Example	Documentation
Learning Path	Build with Vertex AI: Working with Gemini

Setup

Qwiklabs setup

For each lab, you get a new Google Cloud project and set of resources for a fixed time at no cost.

- Make sure you signed into Qwiklabs using an **incognito window**.
- Note the lab's access time (for example, **02:00:00**) and make sure you can finish in that time block.

There is no pause feature. You can restart if needed, but you have to start at the beginning.

- When ready, click **START LAB**.
 - Note your lab credentials. You will use them to sign in to the Google Cloud
- Open Google Console
- Caution:** When you are in the console, do not deviate from the lab instructions. Doing so may cause your account to be blocked. [Learn more.](#)

Console.

Username
google2876526_student@qwiklabs.n

Password
TG959yrKDX

GCP Project ID
qwiklabs-gcp-0855e773352d3560

[New to labs? View our introductory video!](#)

5. Click **Open Google Console**.

6. Click **Use another account** and copy/paste credentials for **this** lab into the prompts.

If you use other credentials, you'll get errors or **incur charges**.

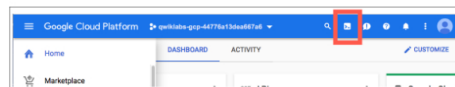
7. Accept the terms and skip the recovery resource page.

Do not click **End Lab** unless you are finished with the lab or want to restart it. This clears your work and removes the project.

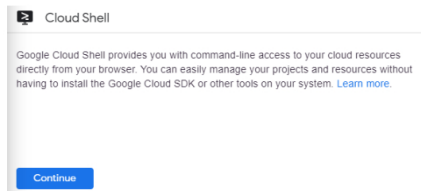
Activate Cloud Shell

Cloud Shell is a virtual machine that is loaded with development tools. It offers a persistent 5GB home directory and runs on the Google Cloud. Cloud Shell provides command-line access to your Google Cloud resources.

In the Cloud Console, in the top right toolbar, click the **Activate Cloud Shell** button.



Click **Continue**.



It takes a few moments to provision and connect to the environment. When you are connected, you are already authenticated, and the project is set to your *PROJECT_ID*. For example:

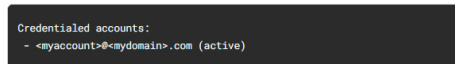


`gcloud` is the command-line tool for Google Cloud. It comes pre-installed on Cloud Shell and supports tab-completion.

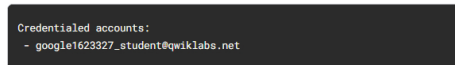
You can list the active account name with this command:

`gcloud auth list`

(Output)



(Example output)



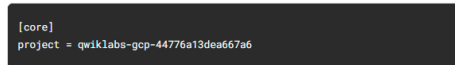
You can list the project ID with this command:

`gcloud config list project`

(Output)



(Example output)



For full documentation of `gcloud` see the [gcloud command-line tool overview](#).

Challenge Scenario



Here is a company overview as provided on Cymbal Shops's website.

Cymbal Shops is an American retail chain headquartered in Minneapolis that sells homewares, electronics, and clothing.

Founded in 1974, Cymbal Shops started out as Cymbal Air, selling AC systems manufactured by Cymbal Group in Minnesota and neighboring states. The company quickly expanded into domestic merchandise in order to satisfy the need for quality products at an affordable price in the midst of a recession.

Cymbal Shops' broad product assortment was once a benefit - capturing planned purchases and last minute splurges. In recent years, however, the company has struggled to adapt to the acceleration in e-commerce. Digitally native companies are on the rise and Cymbal Shops must implement significant changes in order to keep pace and maintain relevance. Today, Cymbal Shops operates 714 stores across North America and reported \$15 billion in revenue in 2019. They currently employ 80,400 employees across the United States and Canada.

Cymbal Shops is a digitally transforming legacy retailer.

It is inspired by clients like: Bed Bath & Beyond, Best Buy, Home Depot, and Nordstrom.

Your Challenge

You work for Cymbal Shops, a company exploring how Gemini's Function Calling can enhance AI app performance. In this lab, you'll learn how to extend large language models (LLMs) with custom code to improve their accuracy and reliability.

LLMs can struggle with tasks like math, string manipulation, or logic-based operations—things that are trivial in code. With Gemini's Function Calling, you can offload those tasks to purpose-built functions. You'll define when and how code should run, extract parameters from user prompts, and return more precise results by blending code execution with LLM reasoning.

You'll build and test this functionality using a Gemini flow, and validate your work through both the Genkit Developer UI and CLI.

Task 1. Open a Jupyter notebook in Vertex AI Workbench

In this task, you will launch a JupyterLab environment using Vertex AI Workbench, create a new Python notebook, and prepare it by installing essential libraries and initializing the Vertex AI SDK.

1. Navigate to **Vertex AI > Workbench**.
2. Under **Instances**, click on **Open Jupyterlab** for **generative-ai-jupyterlab** notebook.

The JupyterLab will run in a new tab.

Filter			
☐	☐	Instance name ↑	Zone
☐	☒	generative-ai-jupyterlab OPEN JUPYTERLAB	us-central1-a

3. On the **Launcher**, under **Notebook**, click on **Python 3** to open a new python notebook.
4. Right-click on the **Untitled.ipynb** file in the menu bar and select **Rename Notebook** the notebook to with the name **function_lab.ipynb**.
5. In the Python Notebook, install the required libraries:

```
! pip3 install --upgrade --quiet --user google-cloud-aiplatform==1.88.0
```

Restart the kernel using the following code so your environment can access the newly installed packages:

```
# Restart kernel after installs so that your environment
can access the new packages
import IPython

app = IPython.Application.instance()
app.kernel.do_shutdown(True)
```

6. In the next cell, run this code snippet to define project variables and initialize the Vertex AI SDK:

```
import vertexai

PROJECT_ID = ! gcloud config get-value project
PROJECT_ID = PROJECT_ID[0]
LOCATION = "us-central1"

print(PROJECT_ID)
```

```
vertexai.init(project=PROJECT_ID, location=LOCATION)
```

7. Finally, import the required libraries:

```
import requests
from vertexai.generative_models import (
    Content,
    FunctionDeclaration,
    GenerationConfig,
    GenerativeModel,
    Part,
    Tool,
)
```

Task 2. Define functions in Python

In this task, you will define custom Python functions, register them as tools using the `FunctionDeclaration` class, and integrate them with a Gemini model using Vertex AI SDK.

Define functions to add and multiply two numbers in Python

Define a function to take two input numbers and **add** them together. Then define another function to **multiply** two inputs. Keep the print statements in the template below in your functions, so you can see when they are being executed.

1. Create **FunctionDeclarations** for your functions. Implement the following **TODOs** and run the cell.

```
# TODO: Define a function to add two numerical inputs and
return the result.
# Keep the print statement within the function.

print("Calling add function")

# TODO: Define a function to multiply two numerical inputs
and return the result.
# Keep the print statement within the function.

print("Calling multiply function")

# TODO: Create FunctionDeclarations for your functions
```

2. Next, define a math Tool with the **add** and **multiply** functions.
3. Define and initialize a model with **gemini-2.0-flash-001**, the **tools**, a **temperature** of 0, and **system instructions** instructing the model to:

```
- Fulfill the user's instructions, including telling jokes.
- If asked to add or multiply numbers, call the provided
functions.
- You may call one function after the other if needed.
- Repeat the result to the user.
```

4. Run the following snippet to initialize the model to start a new chat.

```
chat = model.start_chat()
```

5. Define a function to handle the response from the model.

```
def handle_response(response):

    # If there is a function call then invoke it
    # Otherwise print the response.
    if response.candidates[0].function_calls:
        function_call =
response.candidates[0].function_calls[0]
    else:
        print(response.text)
        return

    # TODO: Complete the following sections
    if # the function_call requests your add function
        # Extract the arguments to use in your function
        # Call your function
        # Send the result back to the chat session with the
model
        # Make a recursive call of this handler function

    elif # the function_call requests your multiply
function
        # Extract the arguments to use in your function
        # Call your function
        # Send the result back to the chat session with the
model
        # Make a recursive call of this handler function

    else:
        # You shouldn't end up here
        print(function_call)
```

6. Save your notebook.

Note: Save the notebook by pressing **Ctrl + S/Command + S**, and then click on the 'Check My Progress' button.

Click **Check my progress** to verify the objective.



Define Python functions to complete tasks and handle model responses

Check my progress

Task 3. Prompt the model to call your functions

In this task, you will evaluate how well your Gemini model responds to various types of user input. Based on the function tools you implemented earlier.

1. Test that the model responds with it's own answer for non-math related questions.

```
response = chat.send_message("Tell me a joke?")
handle_response(response)
```

2. Test the model with the following prompt and print the result to the console: "I have 7 pizzas each with 16 slices. How many slices do I have?"

```
response = chat.send_message("I have 7 pizzas each with 16
slices. How many slices do I have?")
handle_response(response)
```

3. Next, test the model with the following prompt and print the result to the console: "Doug brought 3 pizzas. Andrew brought 4 pizzas. How many pizzas did they bring together?"

```
response = chat.send_message("Doug brought 3 pizzas. Andrew
brought 4 pizzas. How many pizzas did they bring
together?")
handle_response(response)
```

4. Test the model with the following prompt and print the result to the console: "Doug brought 3 pizzas. Andrew brought 4 pizzas. There are 16 slices per pizza. How many slices are there?"

```
response = chat.send_message("Doug brought 3 pizzas. Andrew
brought 4 pizzas. There are 16 slices per pizza. How many
slices are there?")
handle_response(response)
```

5. Test the model with the following prompt and print the result to the console: "Doug brought 4 pizzas, but Andrew dropped 2 on the ground. How many pizzas are left?"

```
response = chat.send_message("Doug brought 4 pizzas, but
Andrew dropped 2 on the ground. How many pizzas are left?")
handle_response(response)
```

6. Save your notebook.

Note: Save the notebook by pressing **Ctrl + S/Command + S**, and then click on the 'Check My Progress' button.

Click **Check my progress** to verify the objective.



Test the model with prompts to see it call your functions and display the results

Check my progress

Congratulations!

You've successfully completed the challenge lab, enhancing an LLM's capabilities by leveraging Gemini Function Calling to delegate code-intensive tasks. This approach involved identifying suitable tasks for code execution, extracting necessary parameters, and utilizing functions to produce superior outputs compared to the LLM's standalone performance.

Manual Last Updated August 21, 2025

Lab Last Tested August 21, 2025

Copyright 2023 Google LLC All rights reserved. Google and the Google logo are trademarks of Google LLC. All other company and product names may be trademarks of the respective companies with which they are associated.



Course Survey

[< Previous](#)

[Next >](#)